

# Kryptologi

Johan Richter

October 7, 2025

# Moderna blockkrypton

Ett modernt blockkrypto kan beskrivas som en funktion,  $E$ , som tar två argument, dels en binär sträng av längd  $n$  som motsvarar klartexten, och dels en binär sträng av längd  $m$  som motsvarar nyckeln, och ut skickar en binär sträng av längd  $n$ , kryptotexten. Om kryptotexten är  $c$ , klartexten är  $m$  och nyckeln är  $k$  kan vi skriva  $c = E(m, k)$ . Ibland används notationen  $c = E_k(m)$ . Den ska uppfylla vissa villkor, t ex

- 1  $E_k$  är en bijektiv funktion för varje värde på  $k$ .
- 2  $E_k$  kan beräknas snabbt.
- 3 Inversen  $D_k = E_k^{-1}$  går snabbt att beräkna.
- 4 Vet man  $E_k(m)$  men inte  $k$  ska det vara svårt att dra slutsatser om  $m$ .
- 5 Vet man  $E_k(m_1), E_k(m_2), \dots$  för några värden av klartexten ska det ändå vara svårt att hitta  $k$ .

Ett teoretiskt optimalt blockkrypto vore att Alice och Bob kom överens om bijektiv avbildning av mängden  $\mathbb{Z}_2^n$  till sig själv, helt slumpmässigt med samma sannolikhet för alla sådana avbildning. Nyckeln vore då en beskrivning av avbildningen. I så fall skulle värdet av  $E_k(m_1)$  inte säga något om  $E_k(m_2)$ , utom att de är olika värden.

Det finns  $(2^n)!$  olika sådana avbildningar, vilket är mer än  $(\frac{2^n}{e})^{2^n}$  stycken. Om blocklängden är 64 bitar hade det krävts mer än  $2^{64}$  bitar för att beskriva nyckeln. Orealistiskt alltså.

I verkligheten för vi nöja oss att  $E$  är en pseudo-slumpmässig funktion.

# DES

DES är ett blockchiffer där blocklängden är 64 bitar och nyckellängden är 56 bitar. Nyckellängden är så kort att med moderna datorer kan man genomföra brute-force attacker, men på 70-talet när den uppfanns var den nog tillräckligt bra.

Konstruktionen av DES använder något som kallas Feistel nätverk som jag börjar med att beskriva.

# Ett Feistel-steg

Anta att bitsträngen vi vill kryptera är  $2n$  bitar lång. Dela upp den i vänstra halvan,  $L$ , och högra halvan  $R$ . Skriv hela strängen som  $(L, R)$ . Låt nyckeln vara  $k$ , som antas ha längd  $m$ . Vi antar vi har en funktion,  $f : \mathbb{Z}_2^n \times \mathbb{Z}_2^m \rightarrow \mathbb{Z}_2^n$ . Denna behöver inte ha någon sorts injektivitetsegenskap.

I ett Feistelsteg skapar vi en ny sträng  $(L', R')$  enligt regeln  $L' = R$  och  $R' = L \oplus f(R, k)$ . Här är  $\oplus$  kompendiets sätt att skriva addition modulo 2. Kan också kallas XOR.

Notera att Feistelsteget är inverterbart, om vi känner till nyckeln. Från  $(L', R')$  får vi  $(L, R)$  med formlerna  $R = L'$  och  $L = R' \oplus f(L', k)$ .

I praktiken, för t ex DES, så har vi en algoritm som från  $k$  härleder vad som kallas för rondnycklar,  $k_1, k_2$ , osv. Ett två-stegs Feistelnätverk utgår från  $(L, R)$  och definierar sedan

$$L_1 = R, \quad R_1 = L \oplus f(R, k_1),$$

$$L_2 = R_1, \quad R_2 = L_1 \oplus f(R_1, k_2).$$

Dett är fortfarande inverterbart. Från  $(L_2, R_2)$  får vi  $(L_1, R_1)$  med formlerna  $R_1 = L_2$  och  $L_1 = R_2 \oplus f(L_2, k_2)$ . Sen går vi tillbaka till  $(L, R)$  på liknande vis. Notera att i  $(L_2, R_2)$  har båda halvor av strängen påverkats av nyckeln.

DES börjar med en fix inledande funktion  $I$ , som kastar om bitar. Om meddelandet är  $m$  så börjar DES räkna  $m_0 = I(m)$ .

Sen gör den 16 rundor av Feistel nätverk och får  $(L_{16}, R_{16})$ .

Sätt  $m_{17} = (R_{16}, L_{16})$ . (Observera ordningen): Kryptogrammet ges av

$$c = I^{-1}(m_{17}).$$

Funktionen  $f$  är beskriven i kompendiet, i detalj. T ex flyttas biten på plats 58 till plats 1. Tycker inte den är intressant nog att beskriva här. Men funktionen  $f$  i varje steg, och hur man hittar nycklarna  $k_1, k_2, \dots, k_{16}$  är intressant.

Jag börjar beskriva  $f$  genom att beskriva algoritmen som räknar den. Jag påminner om att i varje steg räknar vi  $f(R_{i-1}, k_i)$ . Varje rondnyckel kommer vara 48 bitar lång är vad vi behöver veta för att beskriva  $f$ .



Först expanderar vi strängen  $R_{i-1}$  från 32 bitar till 48 med hjälp av tabellen nedan. Den säger att biten på plats 1 i den expanderade strängen är samma som biten på plats 32 i ursprungliga strängen, osv.

|           |           |           |           |           |           |           |           |           |           |           |           |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| <b>1</b>  | <b>2</b>  | <b>3</b>  | <b>4</b>  | <b>5</b>  | <b>6</b>  | <b>7</b>  | <b>8</b>  | <b>9</b>  | <b>10</b> | <b>11</b> | <b>12</b> |
| $b_{32}$  | $b_1$     | $b_2$     | $b_3$     | $b_4$     | $b_5$     | $b_4$     | $b_5$     | $b_6$     | $b_7$     | $b_8$     | $b_9$     |
| <b>13</b> | <b>14</b> | <b>15</b> | <b>16</b> | <b>17</b> | <b>18</b> | <b>19</b> | <b>20</b> | <b>21</b> | <b>22</b> | <b>23</b> | <b>24</b> |
| $b_8$     | $b_9$     | $b_{10}$  | $b_{11}$  | $b_{12}$  | $b_{13}$  | $b_{12}$  | $b_{13}$  | $b_{14}$  | $b_{15}$  | $b_{16}$  | $b_{17}$  |
| <b>25</b> | <b>26</b> | <b>27</b> | <b>28</b> | <b>29</b> | <b>30</b> | <b>31</b> | <b>32</b> | <b>33</b> | <b>34</b> | <b>35</b> | <b>36</b> |
| $b_{16}$  | $b_{17}$  | $b_{18}$  | $b_{19}$  | $b_{20}$  | $b_{21}$  | $b_{20}$  | $b_{21}$  | $b_{22}$  | $b_{23}$  | $b_{24}$  | $b_{25}$  |
| <b>37</b> | <b>38</b> | <b>39</b> | <b>40</b> | <b>41</b> | <b>42</b> | <b>43</b> | <b>44</b> | <b>45</b> | <b>46</b> | <b>47</b> | <b>48</b> |
| $b_{24}$  | $b_{25}$  | $b_{26}$  | $b_{27}$  | $b_{28}$  | $b_{29}$  | $b_{28}$  | $b_{29}$  | $b_{30}$  | $b_{31}$  | $b_{32}$  | $b_1$     |

Sätt sedan  $B = E \oplus k_i$ , där  $E$  är det expanderade resultatet ovan.

Dela in den 48 bitar långa strängen i 8 block om 6 bitar var. Kalla dem  $B_1, B_2$ , osv. Det finns 8 funktioner  $S_1, S_2$ , osv och vi kommer räkna  $S_j(B_j)$  för  $j \in \{1, 2, \dots, 8\}$ .  $S_1$  beskrivs med hjälp av tabellen

|   | 0  | 1  | 2  | 3 | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|---|----|----|----|---|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 14 | 4  | 13 | 1 | 2  | 15 | 11 | 8  | 3  | 10 | 6  | 12 | 5  | 9  | 0  | 7  |
| 1 | 0  | 15 | 7  | 4 | 14 | 2  | 13 | 1  | 10 | 6  | 12 | 11 | 9  | 5  | 3  | 8  |
| 2 | 4  | 1  | 14 | 8 | 13 | 6  | 2  | 11 | 15 | 12 | 9  | 7  | 3  | 10 | 5  | 0  |
| 3 | 15 | 12 | 8  | 2 | 4  | 9  | 1  | 7  | 5  | 11 | 3  | 14 | 10 | 0  | 6  | 13 |

$B_1$  har bitarna  $x_1, x_2, x_3, x_4, x_5, x_6$ .  $x_1x_6$ , läst som binärt tal, ger vilken rad vi ska välja.  $x_2x_3x_4x_5$  ger vilken kolumn vi ska välja. Skriv sedan heltalet du hittar på den platsen med 4 binära siffror. Om  $B_1 = 100111$  så ska vi titta på plats  $(3, 3)$  där det står 2. Alltså är  $S_1(B_1) = 0010$ .

Alla  $S$ -boxarna finns i kompendiet eller på wikipedia. Notera att om vi sätter samman  $S_1(B_1), S_2(B_2), \dots$  så får vi 32 bitar igen. Bitarna permuteras med hjälp av följande tabell:

|    |    |    |    |
|----|----|----|----|
| 16 | 7  | 20 | 21 |
| 29 | 12 | 28 | 17 |
| 1  | 15 | 23 | 26 |
| 5  | 18 | 31 | 10 |
| 2  | 8  | 24 | 14 |
| 32 | 27 | 3  | 9  |
| 19 | 13 | 30 | 6  |
| 22 | 11 | 4  | 25 |

Så biten på plats 1 är gamla biten på plats 16 osv. Resultatet efter denna omkastning av bitarna returneras som  $f(R_{i-1}, k_i)$ .

Återstår att beskriva hur vi genererar nycklarna  $k_1, k_2, \dots$  osv. Vi utgår från en 64-bitars sträng  $K$ . Det är som nyckeln  $k$ , fast vi lagt till en paritetsbit var åttonde bit. Dessa kommer ignoreras nedan, men vi lägger till dem nu för att notationen ska stämma med boken.

Paritetsbitarna kan användas för att upptäcka om fel uppstått i nyckeln vid transporten. Inte en kryptografisk grej, men kanske användbar. Men folk har föreslagit amerikanska staten ville ha kortare nycklar så de kunde angripa chiffret.

Från  $K$  skapar vi en ny sträng med hjälp av följande tabell:

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| 57 | 49 | 41 | 33 | 25 | 17 | 9  |
| 1  | 58 | 50 | 42 | 34 | 26 | 18 |
| 10 | 2  | 59 | 51 | 43 | 35 | 27 |
| 19 | 11 | 3  | 60 | 52 | 44 | 36 |
| 63 | 55 | 47 | 39 | 31 | 23 | 15 |
| 7  | 62 | 54 | 46 | 38 | 30 | 22 |
| 14 | 6  | 61 | 53 | 45 | 37 | 29 |
| 21 | 13 | 5  | 28 | 20 | 12 | 4  |

Den ska tolkas som att den 57:de biten i  $K$  blir nya första biten, 49:de biten i  $K$  blir nya andra biten osv. Dela upp resultatet i två halvor  $C_0$  och  $D_0$  med 28 bitar var.

För  $i = 1, 2, \dots, 16$  räknar vi sedan  $C_i = V_i(C_{i-1})$  och  $D_i = V_i(D_{i-1})$  där  $V_i$  beskriver ett cyklisk shift åt vänster. Antalet steg för  $V_i$  ges av tabellen

| Runda | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|-------|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|
| Skift | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 1 | 2  | 2  | 2  | 2  | 2  | 2  | 1  |

Varje bitsträng  $(C_i, D_i)$  är 56 bitar lång. Från dem väljer vi ut 48 bitar enligt en ny tabell:

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 14 | 17 | 11 | 24 | 1  | 5  |
| 3  | 28 | 15 | 6  | 21 | 10 |
| 23 | 19 | 12 | 4  | 26 | 8  |
| 16 | 7  | 27 | 20 | 13 | 2  |
| 41 | 52 | 31 | 37 | 47 | 55 |
| 30 | 40 | 51 | 45 | 33 | 48 |
| 44 | 49 | 39 | 56 | 34 | 53 |
| 46 | 42 | 50 | 36 | 29 | 32 |

Notera att inga tal upprepas i tabellen.

Den 14:de biten i  $C_i, D_i$  blir alltså första biten i  $k_i$ , osv. Nu vet ni hur nycklarna genereras, och när ni pluggit in hur  $f$  fungerar och hur alla  $S$ -boxarna ser ut kan ni DES utantill.

Tyvärr lite för komplicerat att illustrera med ett exempel som vi räknar för hand.

# Attacker

Bästa attacken mot DES är brute-force. I 1998 byggde en aktivistgrupp en maskin som knäckte DES-krypton på två dygn med brute-force.

Finns också teoretiska attacker. Till exempel differentiell kryptoanalys kallas en attack som använder  $2^{47}$  klartexts-kryptotextspar för att hitta nyckeln.



# Dubbel DES

Ett förslag på förbättrad säkerhet mot brute-force är att kryptera två gånger  $c = E_{k_1}(E_{k_2}(m))$ . Det är ett dåligt förslag.

Anta Eva vet  $m$  och  $c$ . Eva kan göra två separata sökningar över  $E_k(m)$  och  $D_k(c)$  och upptäcka när hon får samma svar på båda. Då hittar hon både  $k_1$  och  $k_2$  på en gång. Kallas för en möte-i-mitten attack.

# Trippel-DES

Följande metoder är bra mot brute-force attacker. Man kan kryptera som  $E_{k_1}(D_{k_2}(E_{k_1}(m)))$ , med användning av två nycklar. Eller så kan man kryptera som  $E_{k_1}(E_{k_2}(E_{k_3}(m)))$  med tre nycklar.

Trippel-DES förlängde livstiden för DES med årtionden, men idag är de för osäkra. Dessutom är blocklängden nog ändå för kort för de flesta användningsområden av block-krypton.

# AES

DES har ersatts av AES (Advanced Encryption Standard). Den har blocklängden 128 och nyckellängd 128. Används kanske varje dag av er dator. Anses vara väldigt säker.

Hur ska vi använda ett blockkrypto om vi har ett säkert sådant och vill kryptera ett meddelanden som består av flera block om  $n$  bitar? Om det finns risk för att ett block upprepas ska vi inte kryptera varje block för sig för då ser Eva när blocken upprepas.

Mitt favoritsätt att använda blockkrypto är *counter mode*. Vi ser det som att  $E_k$  opererar på tal modulo  $2^n$  och vi kommer utföra addition modulo  $2^n$ . Välj ett slumpmässigt tal  $T$  att starta med. Om meddelandet består av blocken  $m_1, m_2, \dots$ , osv så skickar Alice kryptotexten  $T, m_1 \oplus E_k(T), m_2 \oplus E_k(T + 1), \dots$ , osv.

Om  $E$  är ett säkert modern blockkrypto så är counter mode också säkert. Det kan man bevisa, trots att vi inte kan bevisa att säkra blockkrypton finns.